

Basi di dati

Alessandro Carella

Indice

1	Introduzione	3
1.1	Sistema organizzativo, informativo e informatico	3
1.2	Gestione delle informazioni	4
1.3	Dati e informazioni	5
2	Sistemi di gestione di basi di dati	6
2.1	Modelli di dati	8
2.1.1	Schemi e istanze	9
2.1.2	Livelli di astrazione nei DBMS	9
2.1.3	Indipendenza dei dati	10
2.2	Linguaggi e utenti delle basi di dati	10
2.2.1	Linguaggi per basi di dati	11
2.2.2	Utenti e progettisti	11
2.3	Vantaggi e svantaggi dei DBMS	12
3	Metodologie e modelli per il progetto	13
3.1	Il ciclo di vita dei sistemi informativi	13
3.2	Metodologie di progettazione e basi di dati	15
4	Il modello relazionale	18
4.1	Struttura del modello relazionale	18
4.1.1	Modelli logici nei sistemi di basi di dati	18
4.1.2	Relazioni e tabelle	19
4.1.3	Relazioni con attributi	20
4.1.4	Relazioni e basi di dati	21
4.1.5	Informazione incompleta e valori nulli	25
4.2	Vincoli di integrita'	27
4.2.1	Vincoli di tupla	29

4.2.2	Chiavi	29
4.2.3	Chiavi e valori nulli	30
4.2.4	Vincoli di integrita' referenziale	31
5	Algebra relazionale	33
5.0.1	Unione, intersezione e differenza	34
5.0.2	Ridenominazione	35
5.1	Selezione	37
5.2	Proiezione	39
5.3	Join	40
5.3.1	Join naturale	40
5.3.2	Join completi e incompleti	42
5.3.3	Join esterni	44
5.3.4	Semijoin	45
5.3.5	Join n-ario, intersezione e prodotto cartesiano	45
5.3.6	Theta-join ed equi-join	46

Capitolo 1

Introduzione

1.1 Sistema organizzativo, informativo e informatico

DEFINIZIONE 1.1 – «SISTEMA ORGANIZZATIVO»

Insieme di **risorse** e **regole** per lo svolgimento **coordinato** delle **attività** al fine di perseguire degli **scopi**

DEFINIZIONE 1.2 – «**SISTEMA INFORMATIVO**»

Un sistema informativo è:

- **Componente** di una **organizzazione** che gestisce **informazioni** (di interesse)
 - è parte del sistema organizzativo
 - esegue/gestisce processi informativi ovvero processi che coinvolgono informazioni
 - ogni organizzazione ha un sistema informativo, eventualmente non esplicitato nella struttura
 - è spesso di supporto ad altri sottosistemi ed è da studiare nel contesto in cui è inserito
 - è spesso suddiviso in sottosistemi più o meno fortemente integrati

Esiste una dipendenza tra un sistema informativo e un **sistema informatico** per via dell'automatizzazione ed esistono organizzazioni la cui ragion d'essere è la gestione di informazioni e che operano da secoli.

Un sistema informatico è la porzione automatizzata del sistema informativo.

Nel tempo c'è stata un'evoluzione di sistemi informativi verso **razionalizzazione** e **standardizzazione** e delle procedure e dell'organizzazione delle informazioni.

Una risorsa è una qualsiasi fonte o mezzo che valga a fornire aiuto.

1.2 Gestione delle informazioni

Ci sono diverse modalità di gestione delle informazioni:

- idee informali;
- linguaggio naturale;
- disegni, grafici, schemi;
- numeri e codici.

Inoltre, diversi sono i supporti sui quali memorizzare le informazioni:

- memoria umana;
- carta;
- dispositivi elettronici.

Nelle attività standardizzate di sistemi informativi complessi si è giunti ad una organizzazione e codifica, ad esempio nei servizi anagrafici dalle registrazioni discorsive si è giunti a:

- nome e cognome;
- estremi anagrafici;
- codice fiscale.

1.3 Dati e informazioni

Nei sistemi informatici, per ragioni tecnologiche e per la semplicità dei meccanismi di gestione, il concetto di rappresentazione e codifica delle informazioni viene portato all'estremo: le informazioni vengono rappresentate per mezzo di dati che hanno bisogno di essere interpretati per fornire informazioni. È difficile fare una distinzione tra dato e informazione ma per rappresentare in modo approssimativo il concetto possiamo dire che i dati da soli non hanno un significato ben definito ma una volta interpretati correttamente essi forniscono delle informazioni. Quindi l'informazione deriva da un processo di interpretazione e correlazione di dati, tutto ciò che produce variazione nel patrimonio conoscitivo di un soggetto è informazione.

Per fare un esempio di distinzione tra dato e informazione consideriamo la stringa 'BRINDISI 20 26 18', questi sono 4 dati e se correlati alla domanda 'Quali sono le temperature attuali, minime e massime di oggi?' allora i dati possono essere interpretati per fornire informazione e arricchire la conoscenza.

I sistemi informatici si sono limitati a gestire solo i dati e le informazioni da esse estratte.

Capitolo 2

Sistemi di gestione di basi di dati

L'attenzione ai dati ha caratterizzato le applicazioni dell'informatica fin dalle sue origini, ma software dedicati alla loro gestione sono stati realizzati a partire dalla fine degli anni Sessanta.

L'approccio convenzionale alla gestione dei dati sfrutta la presenza di archivi o *file* per memorizzare i dati in modo persistente sulla memoria di massa. Un file consente di memorizzare e ricercare dati, ma fornisce semplici meccanismi di accesso e condivisione. Secondo questo approccio, le procedure scritte in un linguaggio di programmazione sono autonome, ciascuna utilizza uno o più file privati e dati di interesse sono replicati con ridondanza e possibilità di incoerenza. Le basi di dati sono state concepite per superare questi inconvenienti, gestendo in modo integrato e flessibile le informazioni di interesse per diversi soggetti, limitando ridondanza e incoerenza.

Una base di dati è una collezione di dati, di interesse per una o più applicazioni. Un *sistema di gestione di basi di dati* (*Data Base Management System, DBMS*) è un sistema software in grado di gestire collezione di dati che siano *grandi, condivise e persistenti*, assicurando la loro *affidabilità e privacy*. Come ogni prodotto informatico, un DBMS deve essere *efficiente ed efficace*. Una base di dati è una collezione di dati gestita da un DBMS.

Chiariamo con precisione alcune caratteristiche dei DBMS e delle basi di dati:

- le basi di dati sono *grandi*, nel senso che possono avere dimensioni molto maggiori della memoria centrale disponibile, di conseguenza i DBMS devono prevedere una gestione dei dati in memoria secondaria, con architettura molto sofisticate;

- le basi di dati sono *condivise*, nel senso che diversi utenti e applicazioni possono accedervi secondo opportune modalità. In questo modo si riduce la ridondanza dei dati e la conseguente inconsistenza in quanto se esistono copie diverse di uno stesso dato è possibile che queste non siano uguali. Per garantire l'accesso condiviso ai dati, il DBMS dispone di un meccanismo apposito, detto *controllo di concorrenza*;
- le basi di dati sono *persistenti*, cioè hanno un tempo di vita non limitato a quello delle singole esecuzioni dei programmi che le utilizzano;
- i DBMS garantiscono *affidabilità*, cioè la capacità di conservare intatto il contenuto della base di dati in caso di malfunzionamenti hardware o software. A questo scopo i DBMS offrono funzionalità di *salvataggio e ripristino*;
- i DBMS garantiscono la *privatezza* dei dati; ciasun utente, opportunamente riconosciuto, viene abilitato a svolgere solo determinate azioni sui dati, attraverso meccanismi di *autorizzazione*;
- i DBMS sono *efficienti*, cioè sono capaci di svolgere le operazioni utilizzando un insieme di risorse, tempo e spazio, che sia accettabile per gli utenti;
- i DBMS sono *efficaci* in quanto sono capaci di rendere produttive le attività dei loro utenti.

La gestione di collezioni di dati grandi e persistenti è possibile anche con strumenti meno sofisticati dei DBMS, anche con i file, introdotti per gestire insiemi di dati localmente ad una specifica applicazione. I DBMS sono stati concepiti per estendere le funzioni del file system fornendo la possibilità di accesso condiviso agli stessi dati da parte di piu' utenti e applicazioni.

Inoltre, nei programmi tradizionali che accedono a file, ogni programma contiene una descrizione della struttura del file con rischi di incoerenza fra le varie descrizioni e i file stessi, nei DBMS, invece, c'è una porzione di basi di dati, chiamata *dizionario* o *catalogo* che contiene una descrizione centralizzata dei dati che è accessibile dai vari programmi.

Diverse sono le tipologie di basi di dati e DBMS, come:

- basi di dati multimediali;
- data warehouse;

- GIS;
- basi di dati in tempo reale;
- basi di dati attive;
- NoSQL;
- Access;
- Oracle;
- PostgreSQL;
- MySql

e tante altre.

2.1 Modelli di dati

Un *modello di dati* è un insieme di concetti utilizzati per organizzare i dati di interesse e descriverne la struttura in modo che risulti comprensibile a un elaboratore.

Ogni modello di dati fornisce *meccanismi di strutturazione* analoghi ai costruttori di tipo dei linguaggi di programmazione, che permettono di definire nuovi tipi sulla base di tipi predefiniti e costruttori di tipo.

Il *modello relazionale* dei dati, il più diffuso, permette di definire tipi per mezzo del costruttore *relazione*, che consente di organizzare i dati in insiemi di record a struttura fissa. Una relazione viene spesso rappresentata per mezzo di una tabella, le cui righe rappresentano dei record e le colonne corrispondono ai campi dei record, ad esempio:

Docenza	
Corso	NomeDocente
Basi di dati	Rossi
Reti	Neri
Linguaggi	Verdi

Oltre al modello relazionale sono stati definiti altri tipi di modelli:

- il *modello gerarchico* basato sull'uso di strutture ad albero;
- il *modello reticolare* basato sull'uso di grafi;
- il *modello a oggetti*, sviluppato come evoluzione del modello relazionale che estende alle basi di dati il paradigma di programmazione ad oggetti;
- il *modello XML* sviluppato come rivisitazione del modello gerarchico, in cui i dati vengono presentati assieme alla loro descrizione e non devono sottostare rigidamente a un'unica struttura logica;
- modelli semi-strutturati e flessibili, sviluppati nel contesto dei sistemi No-SQL che cercano di superare alcune limitazioni dei sistemi relazionali in termini di prestazioni e di rigidità di organizzazione.

I modelli dei dati vengono detti *logici* per sottolineare il fatto che le strutture utilizzate pur essendo astratte riflettono una particolare organizzazione (ad albero, grafo, ecc.). Più recentemente ai modelli logici, sono stati introdotti i modelli *concettuali* utilizzati per descrivere i dati indipendentemente dalla scelta del modello logico. Il loro nome deriva dal fatto che essi tendono a descrivere i *concetti* del mondo reale e vengono usati nella fase preliminare del processo di progettazione di basi di dati.

2.1.1 Schemi e istanze

Nelle basi di dati esiste una parte invariante nel tempo, detto *schema* della base di dati, costituita dalle caratteristiche dei dati, e una parte variabile nel tempo, detta *istanza* o *stato* della base di dati, costituita dai valori effettivi. Lo schema di una relazione è costituito dalla sua intestazione, ad esempio:

Docenza(Corso, NomeDocente)

Le righe della tabella, invece, variano nel tempo, l'istanza di una relazione è costituita dall'insieme delle sue righe,

2.1.2 Livelli di astrazione nei DBMS

La notazione di modello descritta in precedenza può essere sviluppata tenendo presente di altri fattori nella descrizione dei dati. In particolare, esiste un'architettura standard per DBMS articolata su tre livelli, detti: *esterno*, *logico* e *interno* e per ogni livello esiste uno schema.

- Lo *schema logico* costituisce una descrizione dell'intera base di dati per mezzo del modello logico adottato dal DBMS;
- Lo *schema interno* costituisce la rappresentazione dello schema logico per mezzo di strutture fisiche di memorizzazione, ad esempio una relazione può essere realizzata tramite file sequenziale, file hash o altri approcci;
- Uno *schema esterno* costituisce la descrizione di una porzione della base di dati di interesse, per mezzo del modello logico.

Nei sistemi moderni il livello esterno non è esplicitamente presente, ma è possibile definire relazioni derivate, o *viste*.

2.1.3 Indipendenza dei dati

L'architettura a livelli così definita garantisce l'*indipendenza dei dati* che è la principale proprietà dei DBMS. L'indipendenza dei dati permette a utenti e programmi applicativi che utilizzano una base di dati di interagire a un elevato livello di astrazione, cioè prescinde dai dettagli realizzativi.

L'indipendenza dei dati può essere caratterizzata come indipendenza fisica e logica:

- l'*indipendenza fisica* consente di interagire con il DBMS in modo indipendente dalla struttura fisica dei dati. È possibile modificare le strutture fisiche, come le modalità di organizzazione dei file gestiti dal DBMS, senza influire sulle descrizioni dei dati ad alto livello e quindi sui programmi che utilizzano i dati.
- l'*indipendenza logica* consente di interagire con il livello esterno della base di dati in modo indipendente dal livello logico. Ad esempio, è possibile aggiungere uno schema esterno in base alle esigenze di un nuovo utente o modificare uno schema esterno senza dover modificare lo schema logico e quindi l'organizzazione fisica dei dati.

Gli accessi alla base di dati avvengono solo attraverso il livello esterno ed è il DBMS che traduce le operazioni in termini dei livelli sottostanti.

2.2 Linguaggi e utenti delle basi di dati

I DBMS sono caratterizzati dalla presenza di diversi linguaggi per la gestione dei dati e dalla presenza di molteplici utenti.

2.2.1 Linguaggi per basi di dati

è possibile effettuare diverse operazioni su un DBMS, in particolare quelle relative agli schemi e alle istanze. I linguaggi per basi di dati si distinguono in due categorie:

- *linguaggi di definizione dei dati (Data Definition Language, DDL)*, utilizzati per definire gli schemi logici, esterni e fisici e le autorizzazioni per l'accesso;
- *linguaggi di manipolazione dei dati (Data Manipulation Language, DML)*, utilizzati per l'interrogazione e l'aggiornamento delle istanze di basi di dati.

L'accesso ai dati può essere effettuato con varia modalità:

- tramite linguaggi testuali interattivi, come il linguaggio SQL. Ad esempio, possiamo definire la struttura della relazione *Docenza*, sottoponendo a un interprete SQL la seguente istruzione che appartiene al DDL:

```
create table Docenza (  
    Corso          character (20) ,  
    NomeDocente    character (30)  
)
```

- tramite comandi simili a quelli interattivi immersi in linguaggi di programmazione. Questi linguaggi di programmazione si dicono *linguaggi ospite* perchè ospitando comandi scritti nel linguaggio per basi di dati. Questo approccio si utilizza spesso nella realizzazione di applicazioni parametriche e di farle eseguire ad un utente che non conosce SQL;
- tramite comandi simili a quelli interattivi immersi in linguaggi di sviluppo ad hoc, spesso con funzionalità specifiche;
- tramite interfacce amichevoli che permettono di sintetizzare interrogazioni senza usare un linguaggio testuale.

2.2.2 Utenti e progettisti

Diverse categorie di persone possono interagire con una base di dati o con un DBMS.

- l'*amministratore della base di dati (DataBase Administrator, DBA)* è la persona o gruppo di persone responsabile della progettazione, controllo e amministrazione della base di dati, l'acquisizione di software e risorse hardware e controlla l'efficienza delle operazioni;
- i *progettisti e programmatori di applicazioni* definiscono e realizzano i programmi che accedono alla base di dati. Essi utilizzano i linguaggi DML;
- gli *utenti* utilizzano la base di dati per le proprie attività e gli utenti si dividono in due categorie:
 - *utenti finali o terminalisti* che utilizzano *transazioni*, cioè programmi che realizzano attività predefinite e di frequenza elevata;
 - *utenti casuali* in grado di impiegare i linguaggi interattivi per l'accesso alla base di dati, formulando interrogazioni o aggiornamenti di vario tipo. Essi possono essere specializzati e interagire frequentemente con la base di dati.

2.3 Vantaggi e svantaggi dei DBMS

Dei DBMS possiamo elencare i seguenti vantaggi:

- I DBMS permettono di considerare i dati come una risorsa comune di un'organizzazione, a disposizione di tutte le sue componenti;
- la base di dati fornisce un modello unificato e preciso dalla parte del mondo reale di interesse per l'organizzazione;
- utilizzando un DBMS è possibile un controllo centralizzato dei dati, che può essere arricchito da forme di standardizzazione e beneficiare di "economie di scala";
- la condivisione permette di ridurre ridondanze e inconsistenze;
- l'indipendenza dei dati, fornisce lo sviluppo di applicazioni più flessibili e facilmente modificabili.

In casi di applicazioni con uno o pochi utenti, senza accessi concorrenti possono essere utilizzati file rispetto ai DBMS in quanto può risultare sconveniente.

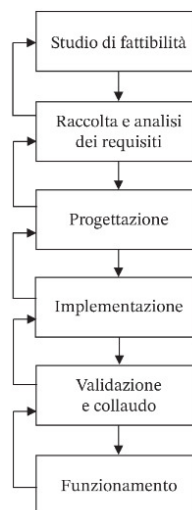
Capitolo 3

Metodologie e modelli per il progetto

3.1 Il ciclo di vita dei sistemi informativi

La progettazione di una basd di dati costituisce solo una delle componenti del processo di sviluppo di un sistema informativo complesso e va quindi inquadrata in un contesto piu' ampio, quello del *ciclo del vita* dei sistemi informativi.

Il ciclo di vita di un sistema informativo comprende le seguenti attività.



- **Studio di fattibilità.** Serve a definire, nel modo piu' preciso possibile, i

costi delle varie alternative possibili e a stabilire le priorità di realizzazione delle varie componenti del sistema.

- **Raccolta e analisi dei requisiti.** Consiste nell'individuazione e nello studio delle proprietà e delle funzionalità che il sistema informativo dovrà avere. Questa fase richiede un'iterazione con gli utenti del sistema e produce una descrizione completa, ma informale, dei dati coinvolti e delle operazioni su di essi. Vengono inoltre stabiliti i requisiti software e hardware del sistema informativo.
- **Progettazione.** Si divide generalmente in *progettazione di dati* e *progettazione delle applicazioni*. Nella prima si individua la struttura e l'organizzazione che i dati dovranno avere, nell'altra si definiscono le caratteristiche dei programmi applicativi.
Le due attività sono complementari e possono procedere in parallelo o in cascata. Le descrizioni dei dati e delle applicazioni prodotti sono formali e fanno riferimento a specifici modelli.
- **Implementazione.** Consiste nella realizzazione del sistema informativo secondo la struttura e le caratteristiche definite nella fase di progettazione. Viene costruita e popolata la base di dati e viene prodotto il codice dei programmi.
- **Validazione e collaudo.** Serve a verificare il corretto funzionamento e la qualità del sistema informativo. La sperimentazione deve prevedere tutte le condizioni operative.
- **Funzionamento.** In questa fase il sistema informativo diventa operativo ed esegue i compiti per i quali era stato progettato. Se non si verificano malfunzionamenti delle funzionalità del sistema, questa attività richiede solo operazioni di gestione e manutenzione.

Il processo non è quasi mai strettamente sequenziale in quanto, durante l'attività di una delle attività citate, bisogna rivedere decisioni prese nell'attività precedente. Inoltre, oltre alle attività descritte, si aggiunge quella di *prototipizzazione*, che consiste nell'uso di specifici strumenti software per la realizzazione rapida di una versione semplificata del sistema informativo, con la quale sperimentare le sue funzionalità. La verifica del prototipo può portare ad un'eventuale revisione del progetto.

I dati hanno un ruolo centrale nei sistemi informativi e questo giustifica uno studio autonomo relativo alla progettazione delle basi di dati.

3.2 Metodologie di progettazione e basi di dati

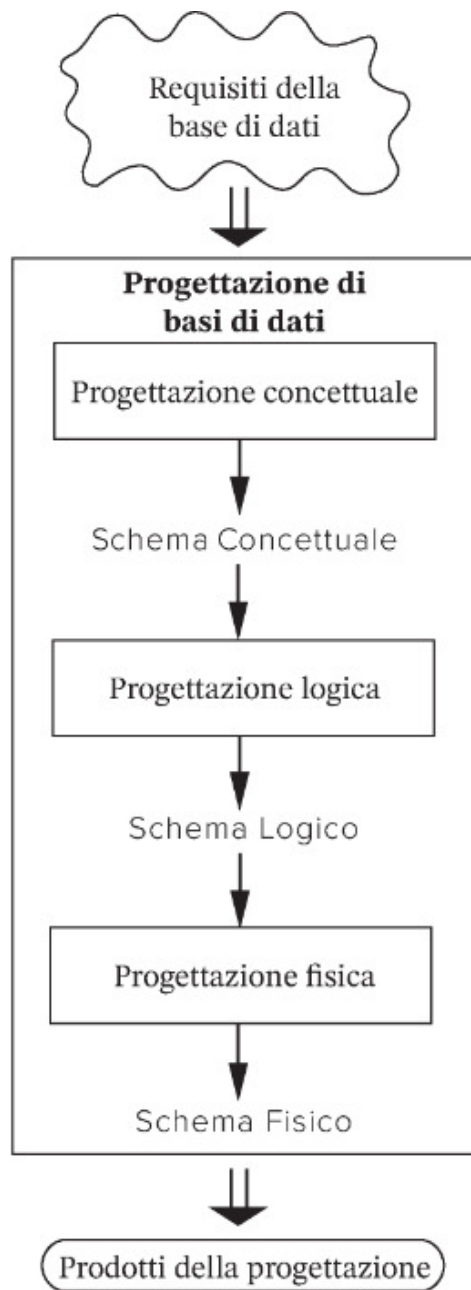
Bisogna precisare cosa si intende per *metodologia di progettazione* e quali sono le proprietà che deve garantire. Una metodologia di progettazione consiste in:

- una *decomposizione* dell'intera attività di progetto in passi successivi indipendenti tra loro;
- una serie di *strategie* da seguire nei vari passi e alcuni *criteri* per la scelta in caso di alternative;
- alcuni *modelli di riferimento* per descrivere i dati di ingresso e uscita delle varie fasi.

Le proprietà che una metodologia deve garantire sono principalmente:

- la *generalità* rispetto alle applicazioni e ai sistemi di gioco (e quindi la possibilità di utilizzo indipendentemente dal problema allo studio e dagli strumenti a disposizione);
- la *qualità del prodotto* in termini di correttezza, completezza ed efficienza rispetto alle risorse impiegate;
- la *facilità d'uso* delle strategie e dei modelli di riferimento.

Nelle basi di dati, nel corso degli anni si è consolidata una metodologia di progetto che è articolata in tre fasi principali da effettuare in cascata. Questa metodologia separa le decisioni relative a *cosa* rappresentare in una base di dati, da quelle relative a *come* farlo.



- **Progettazione concettuale.** Il prodotto di questa fase viene chiamato *schema concettuale* e fa riferimento a un *modello concettuale* dei dati. I modelli concettuali ci consentono di descrivere l'organizzazione dei dati

a un alto livello di astrazione senza tenere conto degli aspetti implementativi. In questa fase, il progettista deve cercare di rappresentare il *contenuto informativo* delle base di dati non pensando a dettagli implementativi. Il modello concettuale piu' noto è il modello **Entity-Relationship**.

- **Progettazione logica.** Consiste nella traduzione dello schema concettuale definito nella fase precedente, adottando il modello di rappresentazione di dati del DBMS a disposizione. Il prodotto di questa fase viene denominato *schema logico* delle base di dati e fa riferimento a un *modello logico* dei dati. Il modello logico ci consente di descrivere i dati prescindendo ancora da dettagli fisici, ma concreta perchè disponibile nei sistemi di gestione di basi di dati. In questa fase, le scelte progettuali si basano su criteri di ottimizzazione delle operazioni da effettuare sui dati. Inoltre, si fa uso di tecniche formali di verifica della qualità dello schema logico ottenuto. Nel caso del modello relazione dei dati, la tecnica usata è quella della *normalizzazione*
- **Progettazione fisica.** In questa fase lo schema logico viene completato con la specifica dei parametri fisici di memorizzazione dei dati. Il prodotto di questa fase viene chiamato *schema fisico* e fa riferimento a un *modello fisico* dei dati che dipende dal DBMS scelto e si basa sui criteri di organizzazione fisica dei dati.

Analizziamo come i requisiti della base di dati vengono utilizzati nelle varie fasi della progettazione. Dobbiamo prima distinguere le *specifiche sui dati* che riguardano il contenuto della base di dati, e *specifiche sulle operazioni* che riguardano l'uso che utenti e applicazioni fanno della base di dati. Nella progettazione concettuale si fa uso delle specifiche sui dati mentre le specifiche sulle operazioni servono solo a verificare che lo schema concettuale sia completo, cioè che contenga le informazioni necessarie per eseguire tutte le operazioni previste.

Capitolo 4

Il modello relazionale

4.1 Struttura del modello relazionale

4.1.1 Modelli logici nei sistemi di basi di dati

Il *modello relazionale* si basa su due concetti fondamentali, quello di *relazione* e *tabella*.

La nozione di *relazione* proviene dalla teoria di insiemi in matematica mentre il concetto di *tabella* è intuitivo.

Il modello relazionale risponde al requisito d'indipendenza dei dati che prevede una distinzione, nella descrizione dei dati, fra il livello *fisico* e il livello *logico*; gli utenti che accedono ai dati e i programmatori che sviluppano le applicazioni fanno riferimento solo al livello logico e i dati descritti al livello logico sono concretamente organizzati per mezzo di strutture fisiche, ma non è necessario conoscerne i dettagli per accedervi. Invece, il modello reticolare e gerarchico, includevano riferimenti alla struttura realizzativa, attraverso l'uso di puntatori e l'ordinamento fisico dei dati.

Il termine *relazione* viene utilizzato in tre accezioni:

1. *relazione matematica* secondo la definizione della teoria degli insiemi;
2. *relazione* secondo la definizione del modello relazionale che presenta differenze a quella della teoria degli insiemi;
3. *relazione* come traduzione di *relationship* costruito del modello concettuale *Entità-Relazione* utilizzato per descrivere legami tra entità del mondo reale.

4.1.2 Relazioni e tabelle

In matematica, dati due insiemi $\mathcal{D}_1$ e $\mathcal{D}_2$, si chiama *prodotto cartesiano* di $\mathcal{D}_1$ e $\mathcal{D}_2$, l'insieme di coppie ordinate (v_1, v_2) , tali che v_1 è un elemento di $\mathcal{D}_1$ e v_2 è un elemento di $\mathcal{D}_2$.

Esempio : Prodotto cartesiano

Dati due insiemi $A = \{1, 2, 3, 4\}$ e $B = \{a, b\}$, il prodotto cartesiano è:

$$A \times B = \{(1, a), (1, b), (2, a), (2, b), (3, a), (3, b), (4, a), (4, b)\}$$

ovvero l'insieme di tutti le possibili coppie dove il primo elemento appartiene ad A e il secondo a B .

Una *relazione matematica* sugli insiemi $\mathcal{D}_1$ e $\mathcal{D}_2$, detti *domini* della relazione, è un sottoinsieme di $\mathcal{D}_1 \times \mathcal{D}_2$.

Le relazioni possono essere espresse graficamente sotto forma tabellare, ad esempio prendendo spunto dall'esempio del **prodotto cartesiano**, possiamo rappresentare la relazione così:

1	a
1	b
2	a
2	b
4	a
4	b

1	a
1	b
4	b

Le definizioni di prodotto cartesiano e relazione date precedentemente hanno fatto riferimento a due insiemi, ma possono essere generalizzate rispetto al numero di insiemi. Dati $n > 0$ insiemi $\mathcal{D}_1, \mathcal{D}_2, \dots, \mathcal{D}_n$, anche non distinti, il prodotto cartesiano di $\mathcal{D}_1 \times \mathcal{D}_2 \times \dots \times \mathcal{D}_n$ è costituito dalle n -uple $(v_1, v_2, \dots, v_n)$ tali che $v_i \in \mathcal{D}_i$ $1 \leq i \leq n$.

Una relazione matematica sui domini $\mathcal{D}_1, \mathcal{D}_2, \dots, \mathcal{D}_n$ è un sottoinsieme del prodotto cartesiano $\mathcal{D}_1 \times \mathcal{D}_2 \times \dots \times \mathcal{D}_n$. Il numero delle n componenti del prodotto cartesiano viene detto *grado* del prodotto cartesiano e della relazione. Il numero di elementi della relazione viene detto *cardinalità* della relazione.

4.1.3 Relazioni con attributi

Una relazione matematica è un insieme di n -uple ordinate $(v_1, v_2, \dots, v_n)$ ove $v_1 \in \mathcal{D}_1, v_2 \in \mathcal{D}_2, \dots, v_n \in \mathcal{D}_n$. Ciascuna n -upla contiene dati fra loro collegati, stabilisce un legame fra loro, ad esempio una n -upla siffatta: "Juventus, Lazio, 3, 1" indica che il risultato della partita tra Juventus e Lazio è 3 a 1.

Una relazione è un insieme quindi:

- non è definito un ordinamento fra le n -uple;
- le n -uple di una relazione sono distinte l'una dall'altra, in quanto tra gli elementi di un insieme non ce ne possono essere due uguali fra loro.

Ciascuna n -upla è, al proprio interno, *ordinata*: l' i -esimo valore di ciascuna proviene dall' i -esimo dominio. L'ordinamento appena descritto corrisponde ad una caratteristica insoddisfacente del concetto di relazione matematica rispetto alla possibilità di organizzare e utilizzare i dati. Infatti, in vari contesti dell'informatica, si tende a privilegiare notazioni *non-posizionali* rispetto a quelli posizionali: si tende a utilizzare quelli posizionali solo quando l'ordinamento corrisponde ad una esigenza intrinseca, come accade nei problemi di natura matematica, in cui gli array permettono di rappresentare in modo ovvio vettori e matrici. Le basi di dati, invece, hanno una struttura che si può ricondurre a quella dei record. Nel caso dei record, ad ogni campo, è associato un nome e allo stesso modo, ad ogni dominio possiamo associare un nome detto *attributo* che descrive il ruolo svolto dal dominio. Per esempio, nella relazione precedente delle squadre, possiamo usare nomi come "SquadraCasa, SquadraOspite, RetiCasa, RetiOspite".

Modificando la definizione di relazione con l'introduzione degli attributi e quindi l'ordinamento degli attributi risulta irrilevante, non è più necessario parlare di primo dominio, secondo dominio ecc.

Per formalizzare i concetti, indichiamo con $\mathcal{D}$ l'insieme dei domini e specifichiamo la corrispondenza fra attributi e domini, nell'ambito di una relazione, per mezzo di una funzione $dom : X \rightarrow \mathcal{D}$, che associa a ciascun attributo $A \in X$ un dominio $dom(A) \in \mathcal{D}$. Poi diciamo che una *tupla* su un insieme di attributi X è una funzione t che associa a ciascun attributo $A \in X$ un valore del dominio $dom(A)$. Quindi una *relazione* su X è un insieme di tuple su X . La differenza nella definizione precedente sta nel fatto che gli elementi delle tuple sono individuati per mezzo degli attributi e non per posizione.

Se t è una tupla su X e A un attributo nell'insieme X allora $t[A]$ indica il valore di t su A , ad esempio:

$$t[SquadraOspite] = Lazio$$

4.1.4 Relazioni e basi di dati

Una base di dati è costituita da piu' relazioni, le cui tuple, contengono valori comuni per stabilire delle corrispondenze. Ad esempio date le seguenti relazioni:

Studenti

Matricola	Cognome	Nome	Data di nascita
276545	Rossi	Maria	25/11/2000
485745	Neri	Anna	23/04/2001
200768	Verdi	Fabio	12/02/2001
587614	Rossi	Luca	10/10/2000
937653	Bruni	Mario	01/12/2000

Esami

Studente	Voto	Corso
276545	28	01
276545	27	04
937653	25	01
200768	24	04

Corsi

Codice	Titolo	Docente
01	Analisi	Giani
03	Chimica	Melli
04	Chimica	Belli

Studenti

Matricola	Cognome	Nome	Data di nascita
276545	Rossi	Maria	25/11/2000
485745	Neri	Anna	23/04/2001
200768	Verdi	Fabio	12/02/2001
587614	Rossi	Luca	10/10/2000
937653	Bruni	Mario	01/12/2000

Esami

Studente	Voto	Corso
276545	28	01
276545	27	04
937653	25	01
200768	24	04

Corsi

Codice	Titolo	Docente
01	Analisi	Giani
03	Chimica	Melli
04	Chimica	Belli

- la relazione *Studenti* contiene informazioni relative a un insieme di studenti, con numero di matricola, cognome, nome e data di nascita;
- la relazione *Esami* contiene informazioni relative a esami: il numero di matricola dello studente, il voto e il corso; questa relazione fa riferimento ai dati contenute nelle altre due, agli studenti con il numero di matricola e ai corsi con il codice di corso;
- la relazione *corsi* contiene relazioni relative ai corsi con il codice del corso, il titolo e il docente.

Questo esempio mostra una delle caratteristiche fondamentali del modello relazionale, il fatto che esso è basato su valori: i riferimenti fra dati in relazioni sono rappresentato per mezzi di valori che compaiono nelle tuple. Rispetto a un modello basato su record e puntatori, il modello relazionale, basato su valori, presenta diversi vantaggi:

- esso richiede di rappresentare solo ciò che è rilevante dal punto di vista dell'applicazione: i puntatori sono qualcosa di aggiuntivo legato ad aspetti realizzativi, nei modelli con puntatori, il programmatore fa riferimento a dati non significativi per l'applicazione;
- la rappresentazione logica dei dati non fa alcun riferimento con quella fisica che può anche subire cambiamenti, il modello relazionale permette di ottenere l'indipendenza fisica dei dati;
- dato che l'informazione è contenuta nei valori, è semplice trasferire i dati da un contesto ad un altro; con i puntatori l'operazione è più complessa perché i puntatori hanno un significato locale al singolo sistema.

Anche in una base di dati relazionale, a livello fisico, i dati possono essere rappresentati con modalità che prevedono l'uso dei puntatori. La differenza rispetto

ai modelli basati su puntatore è che qui i puntatori non sono visibili a modello logico.

Possiamo riassumere le definizioni relative al modello relazionale con più precisione, distinguendo il livello degli schemi a quello delle istanze.

- Uno *schema di relazione* è costituito da un simbolo R , detto *nome della relazione* e da un insieme di attributi $X = \{A_1, A_2, \dots, A_n\}$;
- Uno *schema di base di dati* è un insieme di schemi di relazione con nomi diversi:

$$\mathbf{R} = \{R_1(X_1), R_2(X_2), \dots, R_n(X_n)\}$$

- Un'istanza di relazione o *relazione* su uno schema $R(X)$ è un insieme di r di tuple su X . Talvolta si usa la notazione di $r(X)$;
- Un'istanza di base di dati o semplicemente *base di dati* è un insieme di relazioni $\mathbf{r} = \{r_1, r_2, \dots, r_n\}$ dove ogni r_i per $1 \leq i \leq n$ è uno schema di relazione $R_i(X_i)$

Il modello relazionale permette di rappresentare informazioni strutturate in modo articolato, nella figura sottostante, sono schematizzate tre ricevute emesse da un ristorante. Esse hanno una struttura che prevede alcune informazioni fisse come numero, data e totale e un numero di righe variabile ognuna relativa a un insieme di portate omogenee. Possiamo rappresentare queste informazioni per mezzo di due relazioni, la relazione Ricevute con numero, data e totale e la relazione Dettaglio con quantità, descrizione e importo, associate alla ricevuta stessa tramite il numero.

“DA MARIO”		
Ricevuta n. 1357		
del 5/5/2021		
3	coperti	6,00
2	antipasti	12,00
3	primi	27,00
2	bistecche	36,00
Totale		81,00

“DA MARIO”		
Ricevuta n. 2334		
del 4/7/2021		
2	coperti	4,00
1	antipasti	6,00
2	primi	15,00
2	orate	50,00
2	caffè	3,00
Totale		78,00

“DA MARIO”		
Ricevuta n. 3002		
del 4/8/2021		
3	coperti	6,00
2	antipasti	14,00
3	primi	20,00
1	orate	25,00
1	caprese	8,00
2	caffè	3,00
Totale		76,00

La base di dati che rappresenta le ricevute soprastanti è la seguente:

Ricevute

Num	Data	Totale
1357	5/5/2021	81,00
2334	4/7/2021	78,00
3002	4/8/2021	76,00

Dettaglio

Num	Q.tà	Descr	Costo
1357	3	coperti	6,00
1357	2	antipasti	12,00
1357	3	primi	27,00
1357	2	bistecche	36,00
2334	2	coperti	4,00
2334	1	antipasti	6,00
2334	2	primi	15,00
2334	2	orate	50,00
2334	2	caffè	3,00
3002	3	coperti	6,00
3002	2	antipasti	14,00
3002	3	primi	20,00
3002	1	orate	25,00
3002	1	caprese	8,00
3002	2	caffè	3,00

In questa situazione, la base di dati descritta non tiene traccia della riga dei singoli ordini come accade nelle ricevute, se vogliamo tener traccia dell'impaginazione della ricevuta basta aggiungere un attributo nella relazione Dettaglio che indica la posizione all'interno della ricevuta, come segue:

Dettaglio

Num	Riga	Q.tà	Descr	Costo
1357	1	3	coperti	6,00
1357	2	2	antipasti	12,00
1357	3	3	primi	27,00
1357	4	2	bistecche	36,00
2334	1	2	coperti	4,00
2334	2	1	antipasti	6,00
2334	3	2	primi	15,00
2334	4	2	orate	50,00
2334	5	2	caffè	3,00
3002	1	3	coperti	6,00
3002	2	2	antipasti	14,00
3002	3	3	primi	20,00
3002	4	1	orate	25,00
3002	5	1	caprese	8,00
3002	6	2	caffè	3,00

4.1.5 Informazione incompleta e valori nulli

La struttura del modello relazione è molto semplice e potente. Essa però impone un certo grado di rigidità, in quanto le informazioni devono essere rappresentate per mezzo di tuple di dati omogenee, in ogni relazione possiamo rappresentare solo tuple corrispondenti allo schema di relazione stessa. In molti casi, i dati disponibili possono non corrispondere al formato previsto, per esempio, data la relazione:

Persone(Cognome, Nome, Indirizzo, Telefono)

il valore dell'attributo Telefono potrebbe non essere disponibile per tutte le tuple. Quindi, non sarebbe opportuno utilizzare un valore del dominio per rappresentare l'assenza di informazione. In questo caso, supponendo che i numeri telefonici sono rappresentati per mezzo di numeri interi, potremmo utilizzare lo zero per

indicare l'assenza. In generale, questa scelta non risulta soddisfacente. In primo luogo perchè, richiede l'esistenza di un valore nel dominio mai utilizzato per valori significativi, in secondo luogo l'uso di valori del dominio può generare confusione: la distinzione fra valori veri e fittizi è nascosta e quindi, i programmatori che accedono alla base di dati devono tenerne conto distinguendo opportunamente.

Per rappresentare in modo semplice e comodo l'assenza di informazione, il concetto di relazione viene esteso prevedendo che una tupla possa assumere, su ciascun attributo, o un valore del dominio, o un valore speciale, detto *valore nullo*. Nelle rappresentazioni tabellari il valore nullo viene indicato con il simbolo *null*.

Nei sistemi di basi di dati relazionali è prevista una gestione molto semplice del valore nullo, sul quale non viene fatta alcuna ipotesi e quindi rappresenta il valore senza informazione.

Studenti

Matricola	Cognome	Nome	Data di nascita
276545	Rossi	Maria	null
null	Neri	Anna	23/04/2001
null	Verdi	Fabio	12/02/2001

Esami

Studente	Voto	Corso
276545	28	01
null	27	null
200768	24	null

Corsi

Codice	Titolo	Docente
01	Analisi	Giani
03	Chimica	null
null	Chimica	Belli

Il valore nullo sulla data di nascita è ammissibile perchè si può pensare che l'informazione non sia rilevante, ma un valore nullo sul numero di matricola o sul codice del corso genera problemi in quanto quei valori sono utilizzati per stabilire correlazioni fra tuple di relazioni diverse. Inoltre, la presenza di valori nulli rende inutilizzabili le informazioni. Infine, la presenza di molteplici valori nulli in una relazione può generare dubbi sulla significatività e identità delle tuple. È necessario quindi moderare opportunamente la presenza dei valori nulli nelle nostre relazioni, in genere, quando si definisce una relazione è possibile specifi-

care quali attributi possono assumere valori nulli o meno.

4.2 Vincoli di integrità

In molti casi, non è vero che l'insieme di tuple sullo schema rappresenti informazioni corrette per l'applicazione, dopo aver descritto i problemi relativi alla presenza di valori nulli, analizziamo i problemi relativi alla presenza di valori non nulli. Analizziamo la relazione in figura:

Studenti

Matricola	Cognome	Nome	Data di nascita
200768	Verdi	Fabio	12/02/2001
937653	Rossi	Luca	10/10/2000
937653	Bruni	Mario	01/12/2000

Esami

Studente	Voto	Lode	Corso
200768	36		05
937653	28	lode	01
937653	30	lode	04
276545	25		01

Corsi

Codice	Titolo	Docente
01	Analisi	Giani
03	Chimica	Melli
04	Chimica	Belli

- Nella prima tupla della relazione *Esami* abbiamo un voto pari a 36 che, nel sistema italiano, non è ammissibile;

- Nella seconda tupla della relazione *Esami* viene indicato che è stata attribuita la lode ad un voto di 28;
- Le ultime due tuple della relazione *Studenti* troviamo lo stesso numero di matricola che è impossibile in quanto univoco;
- La quarta tupla della relazione *Esami* presente, per l'attributo **Studente**, un numero di matricola che non compare nella relazione *Studenti*, analogamente compare un codice di corso non presente nella relazione *Corsi*.

In una base di dati, bisogna evitare situazioni di questo tipo, quindi è stato introdotto il concetto di *vincolo di integrità*, come proprietà che deve essere soddisfatta dalle istanze che rappresentano informazioni corrette per l'applicazione. Ogni vincolo può essere visto come un predicato che associa a ogni istanza il valore *vero* o *falso*. In uno schema a base di dati associamo un insieme di predicati e consideriamo corrette le istanze che soddisfano tutti i vincoli. In ciascuno dei casi discussi sopra possiamo introdurre un vincolo che vieti la situazione indesiderata.

Distinguiamo due categorie di vincoli a seconda degli elementi che sono coinvolti in una base di dati.

- Un vincolo è *intrarelazionale* se il soddisfacimento è definito rispetto a singole relazioni della base di dati; i primi tre casi corrispondono a vincoli intrarelazionali. Talvolta, il coinvolgimento riguarda le tuple separatamente le une dalle altre:
 - un *vincolo di tupla* è un vincolo che può essere valutato su ciascuna tupla indipendentemente dalle altre; i primi due casi rientrano in questa categoria.
 - un *vincolo su valori* o *vincolo di dominio* impone una restrizione sul dominio; il primo caso rientra in questa categoria.
- Un vincolo è *interrelazionale* se coinvolge più relazioni; questo è il caso del quarto esempio, in cui la situazione indesiderata può essere vietata richiedendo che un numero di matricola compaia nella relazione *Esami* solo se presente nella relazione *Studenti*.

4.2.1 Vincoli di tupla

I vincoli di tupla esprimono condizioni sui valori di ciascuna tupla indipendentemente dalle altre tuple.

Una possibile sintassi per esprimere condizioni di questo tipo è quella che permette di definire espressioni booleane, con connettivi *and*, *or* e *not* con atomi che confrontano, con operatori di uguaglianza, disuguaglianza e ordinamento, valori di attributo o espressioni aritmetiche su valori di attributo. Ad esempio i vincoli violati negli esempi precedenti possono essere descritti con le espressioni:

$$(Voto \geq 18) \text{ and } (Voto \leq 30)$$

$$(not(Lode = 'lode)) \text{ or } (Voto = 30)$$

Il secondo indica che è ammissibile la lode solo se il voto è pari a 30.

4.2.2 Chiavi

Analizziamo i vincoli di chiave che sono i più importanti del modello relazionale. Negli esempi precedenti il numero di matricola identifica univocamente uno studente, ma ci sono casi in cui anche un insieme di attributi identifica univocamente una tupla.

Quindi, intuitivamente, una chiave è un insieme di attributi utilizzato per identificare univocamente le tuple di una relazione. Per formalizzare la definizione, procediamo in due passi:

1. un insieme di K attributi è *superchiave* di una relazione r se r non contiene due tuple distinte t_1 e t_2 con $t_1[K] = t_2[K]$
2. K è *chiave* di r se è un superchiave minimale di r

Ad esempio, consideriamo la seguente relazione:

Matricola	Cognome	Nome	Nascita	Corso
4328	Rossi	Luigi	29/04/2000	Ing. Informatica
6328	Rossi	Dario	29/04/2000	Ing. Informatica
4766	Rossi	Luca	01/05/2001	Ing. Civile
4856	Neri	Luca	01/05/2001	Ing. Meccanica
5536	Neri	Luca	05/03/1999	Ing. Meccanica

- l'insieme $\{Matricola\}$ è superchiave ed è anche una superchiave minimale in quanto contiene un solo attributo;

- l'insieme $\{Cognome, Nome, Nascita\}$ è superchiave e nessuno dei suoi sottoinsiemi è superchiave; infatti esistono due tuple uguali su Cognome e Nascita, due uguali su Cognome e Nome e due uguali su Nome e Nascita;
- l'insieme $\{Matricola, Corso\}$ è superchiave, ma non è superchiave minimale perchè esiste un suo sottoinsieme proprio $\{Matricola\}$, che è superchiave minimale, e quindi $\{Matricola, Corso\}$ non è una chiave;
- l'insieme $\{Nome, Corso\}$ non è superchiave perchè nella relazione compaiono due tuple uguali su Nome e Corso.

Possiamo notare come ciascuna relazione e ciascuno schema di relazione abbiano sempre una chiave. Una relazione è un insieme e quindi, è costituita da elementi diversi tra loro; di conseguenza per ogni relazione $r(X)$, l'insieme X di attributi è una superchiave per la relazione. Ci possono essere due possibili casi: o l'insieme è anche chiave, oppure non è chiave, perchè esiste un'altra superchiave in esso contenuta; allora si può ripetere lo stesso ragionamento per questo sottoinsieme e così via.

Il fatto che su ogni schema di relazione possa essere definita almeno una chiave garantisce l'accessibilità a tutti i valori di una base di dati e la loro univoca identificabilità. Inoltre, permette di stabilire efficacemente quelle corrispondenze fra dati contenuti in relazioni diverse che caratterizzano il modello relazionale come modello basato su valori. I valori attraverso cui vengono stabilite le corrispondenze fra tuple di relazioni diverse sono valori delle chiavi delle relazioni cui si fa riferimento dall'esterno.

4.2.3 Chiavi e valori nulli

Consideriamo la relazione in figura:

Matricola	Cognome	Nome	Nascita	Corso
null	Rossi	Mario	null	Ing. Informatica
4766	Rossi	Luca	01/05/2001	Ing. Civile
4856	Neri	Luca	null	null
null	Neri	Luca	05/03/1999	Ing. Civile

Esaminando questa relazione, con due chiavi Matricola e (Cognome, Nome, Nascita), notiamo problemi di due tipi: la prima tupla ha valori nulli su Matricola e Nascita e quindi questa tupla non è identificabile in alcun modo. Inoltre, non

è possibile in altre relazioni, fare riferimento a questa tupla, visto che andrebbe fatto con il valore di una chiave. Anche le ultime due tuple presentano un problema: nonostante ciascuna delle due tuple abbia una chiave completamente specificata, la presenza di valori nulli rende impossibile capire se le due tuple facciano riferimento allo stesso studente.

Questo esempio suggerisce la necessità di porre dei limiti alla presenza di valori nulli nelle chiavi delle relazioni. In pratica, si adotta una soluzione semplice, che permette di garantire l'identificazione univoca di tutte le tuple e la possibilità di far riferimento a esse: su una delle chiavi, detta *chiave primaria*, si vieta la possibilità di avere valori nulli, sulle altre i valori nulli sono ammessi. Gli attributi che costituiscono la chiave primaria vengono evidenziati attraverso la sottolineatura. La maggior parte dei riferimenti tra relazioni avviene attraverso la chiave primaria.

In quasi tutti i casi reali è possibile trovare attributi i cui valori sono identificanti e sempre disponibili. Se questo non accade è necessario introdurre un attributo aggiuntivo, solitamente un codice, che viene generato e assegnato ad ogni tupla all'atto dell'inserimento.

4.2.4 Vincoli di integrità referenziale

Per discutere la più importante classe di vincoli interrelazionali, consideriamo la base di dati in figura:

Infrazioni

<u>Codice</u>	<u>Data</u>	<u>Agente</u>	<u>Articolo</u>	<u>Stato</u>	<u>Numero</u>
143256	25/10/2021	567	44	I	AB 234 ZK
987554	26/10/2021	456	34	I	AB 234 ZK
987557	26/10/2021	456	34	I	CB 123 AA
630876	15/10/2021	456	53	F	CB 123 AA
539856	12/10/2021	567	44	F	CB 123 AA

Agenti

<u>Matricola</u>	<u>CF</u>	<u>Cognome</u>	<u>Nome</u>
567	RSSM ...	Rossi	Mario
456	NREL ...	Neri	Luigi
638	NREP ...	Neri	Piero

Auto

<u>Stato</u>	<u>Numero</u>	<u>Proprietario</u>	<u>Indirizzo</u>
I	CB 123 AA	Verdi Piero	Via Tigli ...
I	DE 834 ZZ	Verdi Piero	Via Tigli ...
I	AB 234 ZK	Bini Luca	Via Aceri ...
F	CB 123 AA	Beau Marcel	Rue Louis XIV ...

La prima relazione, contiene informazioni relative a un insieme di infrazioni del codice della strada, la seconda agli agenti di polizia che le hanno rilevate e la terza a un insieme di autoveicoli di Paesi Europei. Le informazioni delle *Infrazioni* sono rese significative attraverso il riferimento alla relazione *Agenti* per mezzo della matricola di essi e alla relazione *Auto* per tramite degli attributi Stato e Numero. I riferimenti sono significativi in quanto i valori nella relazione *Infrazioni* sono uguali a quelli presenti nelle altre due relazioni.

Un *vincolo di integrità referenziale* o *foreign key* fra un insieme di attributi X di una relazione R_1 e un'altra relazione R_2 è soddisfatto se i valori di X di ciascuna tupla dell'istanza di R_1 compaiono come valori della chiave dell'istanza di R_2 . Questa definizione richiede particolare attenzione, nei casi in cui la relazione alla quale ci riferiamo è composta da più attributi e nel caso in cui vi siano più chiavi.

Vedendo prima il caso in cui la chiave di R_2 è unica e composta da un solo attributo B allora il vincolo di integrità referenziale fra l'attributo di A di R_1 e la relazione R_2 è soddisfatto se, per ogni tupla t_1 in R_1 per cui $t_1[A]$ non è nullo, esiste una tupla t_2 in R_2 tale che $t_1[A] = t_2[B]$. Nel caso più generale, dobbiamo fare attenzione al fatto che ciascuno degli attributi X deve corrispondere a un preciso attributo della chiave K in R_2 . È necessario specificare un ordinamento sia nell'insieme X sia di K . Indicando gli attributi $X = A_1A_2 \dots A_n$ e $K = B_1B_2 \dots B_n$ il vincolo è soddisfatto se per ogni tupla t_1 in R_1 senza nulli su X esiste una tupla t_2 in R_2 con $t_1[A_i] = t_2[B_i] \quad \forall i \in [1, n]$.

Una considerazione va fatta riguardo le relazioni con più chiavi. In questo caso, è opportuno identificarne una come chiave primaria e riferimenti vengono fatti attraverso essa. Però, va notato che non tutti i sistemi di gestione di base di dati permettono di indicare esplicitamente la chiave primaria. In questi casi, il vincolo di integrità referenziale deve indicare esplicitamente gli attributi che compongono la chiave cui si fa riferimento.

Capitolo 5

Algebra relazionale

Come detto precedentemente, i linguaggi per basi di dati si suddividono in due categorie.

- Operazioni sullo schema:
 - **DDL: Data Definition Language;**
- Operazioni sui dati:
 - **DML: Data Manipulation Language**

I linguaggi di interrogazione per basi di dati relazionali, invece, possono essere:

- **dichiarativi** che specificano le proprietà del risultato quindi cosa si vuole ottenere;
- **procedurali** che specificano le modalità di generazione del risultato quindi come ottenere il risultato.

I linguaggi di interrogazione possono essere:

- **Algebra relazione** che è di tipo procedurale;
- **Calcolo relazionale** che è dichiarativo e teorico;
- **SQL (Structed Query Language)** che è parzialmente dichiarativo (reale);
- **QBE (Query By Example)** che è di tipo dichiarativo (reale).

In questo capitolo analizzeremo l'algebra relazione che come abbiamo detto è un linguaggio di tipo procedurale basato su concetti di tipo algebrico. Esso è costituito da un insieme di operatori, definiti su relazioni e che producono ancora relazioni come risultati ed è possibile anche formare espressioni che coinvolgono più operatori allo scopo di formulare anche interrogazioni più complesse. Gli operatori sono:

- operatori insiemistici tradizionali: *unione*, *intersezione* e *differenza* che possono essere anche definiti, con qualche differenza, sulle relazioni;
- operatori di *ridenominazione*, *selezione* e *proiezione* che sono più specifici;
- operatore di *join* in varie forme: *join naturale*, *prodotto cartesiano*, *semijoin* e *theta-join*;
- operatore di *divisione*, che è ridondante perché esprimibile per mezzo di altri operatori.

5.0.1 Unione, intersezione e differenza

Notiamo che le relazioni sono insiemi e quindi ha senso definire su di esse gli operatori insiemistici tradizionali di unione, differenza e intersezione, quest'ultima è esprimibile per mezzo della differenza in quanto $r \cap s = r - (r - s)$. Però, dobbiamo prestare attenzione sul fatto che una relazione non è genericamente un insieme di tuple, ma un insieme di tuple *omogenee*, quindi pur potendo, il linea di principio, definire delle operazioni insiemistiche sulle relazioni, non ha senso, dal punto di vista del modello relazionale, definirli con riferimento a relazioni su attributi diversi. Pertanto, consideriamo ammissibili, nell'algebra relazionale, solo applicazioni degli operatori di unione, differenza e intersezione a coppie di operandi definite sugli stessi attributi. La seguente figura mostra degli esempi di applicazione dei tre operatori, che confermano le classiche definizioni ma ridefinite sul nostro contesto:

- l'*unione* di due relazioni r_1 e r_2 definite sullo stesso insieme di attributi X è indicata con $r_1 \cup r_2$ ed è una relazione su X contenente le tuple che appartengono a r_1 oppure a r_2 o a entrambe;
- l'*intersezione* di $r_1(X)$ e $r_2(X)$ è indicata con $r_1 \cap r_2$ ed è una relazione su X contenente le tuple che appartengono sia a r_1 che a r_2 ;

- la *differenza* di r_1 e r_2 è indicata con $r_1 - r_2$ ed è una relazione su X contenente le tuple che appartengono a r_1 e non appartengono a r_2 .

Laureati

Matricola	Cognome	Età
7274	Rossi	37
7432	Neri	39
9824	Verdi	38

Dirigenti

Matricola	Cognome	Età
9297	Neri	56
7432	Neri	39
9824	Verdi	38

Laureati $\cup$ Dirigenti

Matricola	Cognome	Età
7274	Rossi	37
7432	Neri	39
9824	Verdi	38
9297	Neri	56

Laureati $\cap$ Dirigenti

Matricola	Cognome	Età
7432	Neri	39
9824	Verdi	38

Laureati $-$ Dirigenti

Matricola	Cognome	Età
7274	Rossi	37

5.0.2 Ridenominazione

La limitazione imposta agli operatori insiemistici, anche se giustificata, risulta molto restrittiva. Consideriamo infatti le relazioni in figura:

Paternità

Padre	Figlio
Adamo	Caino
Adamo	Abele
Abramo	Isacco
Abramo	Ismaele

Maternità

Madre	Figlio
Eva	Caino
Eva	Set
Sara	Isacco
Agar	Ismaele

Paternità $\cup$ Maternità ??

Sarebbe sensato eseguire su di esse una sorta di unione per avere tutte le coppie genitore-figlio ma questo non è possibile, perchè gli attributi sono diversi, in una

troviamo l'attributo *Padre* e nell'altra troviamo l'attributo *Madre*. Per risolvere il problema utilizziamo un operatore che ha lo scopo di adeguare il nome degli attributi in base alle necessità. Questo si chiama operatore di *ridenominazione* che cambia il nome degli attributi lasciando invariato il contenuto delle relazioni. Consideriamo le relazioni in figura:

Paternità		$\rho_{\text{Genitore} \leftarrow \text{Padre}}(\text{Paternità})$	
Padre	Figlio	Genitore	Figlio
Adamo	Caino	Adamo	Caino
Adamo	Abele	Adamo	Abele
Abramo	Isacco	Abramo	Isacco
Isacco	Giacobbe	Isacco	Giacobbe

Come possiamo notare, la ridenominazione agisce solo sullo schema, cambiando il nome dell'attributo *Padre* in *Genitore* e si denota con:

$$\rho_{\text{Genitore} \leftarrow \text{Padre}}$$

Ora definiamo questo operatore in forma generale. Sia r una relazione definita su un insieme di attributi X e sia Y un altro insieme di attributi con la stessa cardinalità. Siano $A_1, A_2, \dots, A_k$ e $B_1, B_2, \dots, B_k$ rispettivamente un ordinamento per gli attributi in X e un ordinamento per quelli in Y . Allora la ridenominazione:

$$\rho_{B_1 B_2 \dots B_k \leftarrow A_1 A_2 \dots A_k}(r)$$

contiene una tupla t' per ciascuna tupla t in r , definita come segue: t' è una tupla su Y e $t'[B_i] = t[A_i] \quad \forall i = 1, \dots, k$. La definizione conferma che ciò che cambia sono i nomi degli attributi, mentre i valori rimangono inalterati e vengono associati ai nuovi attributi. In pratica, nelle due liste $A_1 A_2 \dots A_k$ e $B_1 B_2 \dots B_k$ indicheremo solo gli attributi che vengono ridenominati, cioè quelli per cui $A_i \neq B_i$. Quindi ritornando alle relazioni precedenti, l'unione delle due relazioni è la seguente:

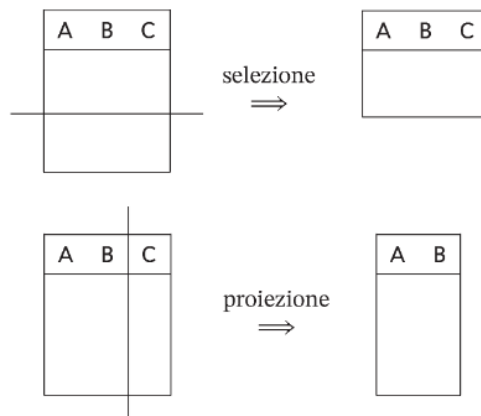
$$\rho_{\text{Genitore} \leftarrow \text{Padre}}(\text{Paternità}) \cup \rho_{\text{Genitore} \leftarrow \text{Madre}}(\text{Maternità})$$

Genitore	Figlio
Adamo	Caino
Adamo	Abele
Abramo	Isacco
Abramo	Ismaele
Eva	Caino
Eva	Set
Sara	Isacco
Agar	Ismaele

5.1 Selezione

Esaminiamo, ora, gli operatori più tipici dell'algebra relazionale, che permettono di manipolare le relazioni. Si tratta di tre operatori: selezione, proiezione e join con diverse varianti.

È bene precisare che la selezione e la proiezione svolgono funzioni “complementari”, sono entrambe definite su un operando e producono come risultato una porzione dell'operando. Più precisamente la selezione produce un sottoinsieme delle tuple, su tutti gli attributi, mentre la proiezione dà un risultato cui contribuiscono tutte le tuple, ma su un sottoinsieme degli attributi. Quindi, possiamo dire che la selezione genera “decomposizioni orizzontali” e la proiezione “decomposizioni verticali”. La situazione descritta è mostrata nella seguente figura:



La selezione è denotata dal simbolo σ a pedice del quale viene indicata la “condizione di selezione” opportuna. Il risultato contiene le tuple dell’operando che soddisfano la condizione di selezione. Le condizioni di selezione possono prevedere confronti tra attributi e confronti tra attributi e costanti e possono essere ottenute combinando condizioni semplici con i connettivi logici: $\vee$ (or), $\wedge$ (and) e $\neg$ (not).

In termini più precisi data una relazione $r(X)$, una *formula proposizionale* F su X è una formula ottenuta combinando, con i connettivi $\vee, \wedge$ e $\neg$, condizioni atomiche del tipo $A \Theta B$ o $A \Theta c$, dove:

- Θ è un operatore di confronto ($=, \neq, <, >, \leq, \geq$);
- A e B sono attributi in X sui cui valori il confronto Θ abbia senso;
- c è una costante con il dominio di A .

Seguono due esempi di selezione:

Cittadini

Cognome	Nome	CittàDiNascita	Residenza
Rossi	Mario	Roma	Milano
Neri	Luca	Roma	Roma
Verdi	Nico	Firenze	Firenze
Rossi	Marco	Napoli	Firenze

$\sigma_{\text{CittàDiNascita}=\text{Residenza}}(\text{Cittadini})$

Cognome	Nome	CittàDiNascita	Residenza
Neri	Luca	Roma	Roma
Verdi	Nico	Firenze	Firenze

Impiegati

Cognome	Nome	Età	Stipendio
Rossi	Mario	25	2000,00
Neri	Luca	40	3000,00
Verdi	Nico	36	4500,00
Rossi	Marco	40	3900,00

$\sigma_{\text{Età}>30 \wedge \text{Stipendio}>4000,00}(\text{Impiegati})$

Cognome	Nome	Età	Stipendio
Verdi	Nico	36	4500,00

Date una formula F e una tupla t , è definito un valore di verità per F su t :

- $A\Theta B$ è vera su t se $t[A]$ è in relazione Θ con $t[B]$, altrimenti è falsa;
- $A\Theta c$ è vera su t se $t[A]$ è in relazione Θ con c , altrimenti è falsa;
- $F_1 \vee F_2, F_1 \wedge F_2, \neg F_1$ hanno l'usuale significato.

Possiamo quindi completare la definizione di selezione: $\Theta_F(r)$, in cui r è una relazione e F è una formula proposizionale, produce una relazione sugli stessi attributi di r che contiene le tuple di r su cui F è vera.

Seguono le proprietà dell'operatore di selezione:

- *Cardinalità*: $|\Theta_F(r)| \leq |r|$
- *Grado*: $\text{grado}(\Theta_F(r)) = \text{grado}(r)$
- *Proprietà commutativa*: $\Theta_{F_1}(\Theta_{F_2}(r)) = \Theta_{F_2}(\Theta_{F_1}(r))$
- *Idempotenza*: $\Theta_{F_1}(\Theta_{F_2}(r)) = \Theta_{F_1 \wedge F_2}(r)$

5.2 Proiezione

L'operatore di proiezione presenta una definizione ancora più semplice di quella di selezione: dati una relazione $r(X)$ e un sottoinsieme Y di X , la *proiezione* di r su Y è l'insieme di tuple su Y ottenute dalle tuple di r considerando solo i valori su Y :

$$\pi_Y(r) = \{t[Y] \mid t \in r\}$$

Seguono due esempi di proiezione:

Impiegati				$\pi_{\text{Cognome, Nome}}(\text{Impiegati})$	
Cognome	Nome	Reparto	Capo	Cognome	Nome
Rossi	Mario	Vendite	Gatti	Rossi	Mario
Neri	Luca	Vendite	Gatti	Neri	Luca
Verdi	Mario	Personale	Lupi	Verdi	Mario
Rossi	Marco	Personale	Lupi	Rossi	Marco

In questo esempio viene illustrato chiaramente il concetto citato.

Impiegati				$\pi_{\text{Reparto, Capo}}(\text{Impiegati})$	
Cognome	Nome	Reparto	Capo	Reparto	Capo
Rossi	Mario	Vendite	Gatti	Vendite	Gatti
Neri	Luca	Vendite	Gatti	Vendite	Gatti
Verdi	Mario	Personale	Lupi	Personale	Lupi
Rossi	Marco	Personale	Lupi	Personale	Lupi

In questo esempio, invece, troviamo un numero di tuple inferiore a quelle dell'operando, perchè alcune tuple, avendo valori uguali sugli attributi della proiezione, danno lo stesso contributo alla proiezione. Essendo le relazioni definite come insiemi, non possono apparire più tuple uguali fra loro. Esiste, quindi, un legame fra i vincoli di chiave e le proiezioni relativamente a questo problema: $\pi_Y(r)$ contiene lo stesso numero di tuple di r se e solo se Y è superchiave per r . Infatti:

- se Y è superchiave, allora r non contiene tuple uguali su Y , quindi ogni tupla dà un contributo diverso alla proiezione;
- se la proiezione ha tante tuple quante l'operando, allora ciascuna tupla contribuisce alla proiezione con valori diversi, quindi r non contiene coppie di tuple uguali su Y e questa è la definizione di superchiave.

5.3 Join

Esaminiamo l'operatore più caratteristico dell'algebra relazionale, l'operatore di join, che permette di correlare dati contenuti in relazioni diverse, confrontando i valori contenuti in esse e utilizzando la caratteristica fondamentale del modello, cioè quella di essere basato su valori. Esistono due versioni dell'operatore: il *join naturale* utile per riflessioni astratte e il *theta-join* che è più rilevante dal punto di vista pratico.

5.3.1 Join naturale

Il *join naturale* è un operatore che correla dati in relazioni diverse, sulla base di valori uguali in attributi con lo stesso nome. L'operatore è denotato con il simbolo $\bowtie$. Il risultato del join è costituito da una relazione sull'unione degli insiemi di attributi degli operandi e le sue tuple sono ottenute combinando le tuple degli operandi con valori uguali sugli attributi comuni. Un esempio di join è mostrato nella seguente figura:

r_1	Impiegato	Reparto	r_2	Reparto	Capo
	Rossi	vendite		produzione	Mori
	Neri	produzione		vendite	Bruni
	Bianchi	produzione			
$r_1 \bowtie r_2$					
	Impiegato	Reparto		Capo	
	Rossi	vendite		Bruni	
	Neri	produzione		Mori	
	Bianchi	produzione		Mori	

In generale, il join naturale $r_1 \bowtie r_2$ di $r_1(X_1)$ e $r_2(X_2)$ è una relazione definita su $X_1 X_2$ come segue:

$$r_1 \bowtie r_2 = \{t \text{ su } X_1 X_2 \mid \text{esistono } t_1 \in r_1 \text{ e } t_2 \in r_2 \text{ con } t[X_1] = t_1 \text{ e } t[X_2] = t_2\}$$

Più sinteticamente, possiamo scrivere:

$$r_1 \bowtie r_2 = \{t \text{ su } X_1 X_2 \mid t[X_1] \in r_1 \text{ e } t[X_2] \in r_2\}$$

La definizione conferma che le tuple del risultato sono ottenute combinando tuple degli operandi con valori uguali su attributi comuni. Il grado della relazione ottenuta come risultato di un join è minore o uguale della somma dei gradi dei due operandi, perchè gli attributi omonimi degli operandi compaiono una sola volta nel risultato.

Molto frequentemente, si eseguono join sulla base di valori di una chiave delle relazioni coinvolte, esplicitando i riferimenti fra tuple. In molti di questi casi, è anche definito, fra gli attributi coinvolti, un vincolo di integrità referenziale. Consideriamo la relazione in figura:

Infrazioni	Codice	Data	Agente	Articolo	Stato	Numero	
	143256	25/10/2021	567	44	I	AB 234 ZK	
	987554	26/10/2021	456	34	I	AB 234 ZK	
	987557	26/10/2021	456	34	I	CB 123 AA	
	630876	15/10/2021	456	53	F	CB 123 AA	
	539856	12/10/2021	567	44	F	CB 123 AA	
Auto	Stato	Numero	Proprietario	Indirizzo			
	I	CB 123 AA	Verdi Piero	Via Tigli ...			
	I	DE 834 ZZ	Verdi Piero	Via Tigli ...			
	I	AB 234 ZK	Bini Luca	Via Aceri ...			
	F	CB 123 AA	Beau Marcel	Rue Louis ...			
Infrazioni $\bowtie$ Auto							
Codice	Data	Ag	Art	Stato	Numero	Proprietario	Indirizzo
143256	25/10/2021	567	44	I	AB 234 ZK	Bini Luca	Via Aceri ...
987554	26/10/2021	456	34	I	AB 234 ZK	Bini Luca	Via Aceri ...
987557	26/10/2021	456	34	I	CB 123 AA	Verdi Piero	Via Tigli ...
630876	15/10/2021	456	53	F	CB 123 AA	Beau Marcel	Rue Louis ...
539856	12/10/2021	567	44	F	CB 123 AA	Beau Marcel	Rue Louis ...

Notiamo che ciascuna delle tuple di *Infrazioni* è stata combinata con una e una sola delle tuple di *Auto*: una sola perchè **Prov** e **Numero** formano una chiave di

Auto, almeno una perchè è definito il vincolo di integrità referenziale fra **Prov** e **Numero** in *Infrazioni* e *Auto*. Il join, quindi, ha esattamente tante tuple quante la relazione *Infrazioni*.

5.3.2 Join completi e incompleti

Analizziamo diversi esempi di join, in modo da effettuare delle valutazioni sulla dimensione del risultato e al contributo a esso portato dalle tuple degli operandi. Nel primo esempio di join mostrato, ciascuna tupla di ciascuno degli operandi contribuisce ad almeno una tupla del risultato, quindi ci troviamo in un caso di *join completo*: per ogni tupla t_1 di r_1 , esiste una tupla t in $r_1 \bowtie r_2$ tale che $t[X_1] = t_1$, analogamente per r_2 . Questa proprietà non è sempre verificata, perchè richiede una corrispondenza fra le tuple delle due relazioni.

Consideriamo la relazione in figura:

r_1	Impiegato	Reparto	r_2	Reparto	Capo
	Rossi	vendite		produzione	Mori
	Neri	produzione		acquisti	Bruni
	Bianchi	produzione			

$r_1 \bowtie r_2$	Impiegato	Reparto	Capo
	Neri	produzione	Mori
	Bianchi	produzione	Mori

Figura 5.1: Un join con tuple *dangling*

La figura mostra un join in cui alcune tuple degli operandi non contribuiscono al risultato, perchè l'altra relazione non contiene tuple con gli stessi valori sull'attributo comune. Queste tuple vengono chiamate *dangling*. Inoltre, è possibile che nessuna tupla degli operandi sia combinabile, e allora il risultato del join è la relazione vuota.

r_1	Impiegato	Reparto	r_2	Reparto	Capo
	Rossi	vendite		concorsi	Mori
	Neri	produzione		acquisti	Bruni
	Bianchi	produzione			

$r_1 \bowtie r_2$	Impiegato	Reparto	Capo

Figura 5.2: Un join vuoto

All'estremo opposto, è possibile che ciascuna delle tuple di ciascuno degli operandi sia combinabile con tutte le tuple dell'altro. In tal caso, il risultato contiene un numero di tuple pari al prodotto della cardinalità degli operandi e cioè $|r_1| \times |r_2|$.

r_1	Impiegato	Progetto	r_2	Progetto	Capo
	Rossi	A		A	Mori
	Neri	A		A	Bruni
	Bianchi	A			

$r_1 \bowtie r_2$	Impiegato	Reparto	Capo
	Rossi	A	Mori
	Neri	A	Mori
	Bianchi	A	Mori
	Rossi	A	Bruni
	Neri	A	Bruni
	Bianchi	A	Bruni

Figura 5.3: Un join con $|r_1| \times |r_2|$ tuple

Quindi il join di r_1 e r_2 contiene un numero di tuple compreso fra 0 e $|r_1| \times |r_2|$. Inoltre:

- se il join di r_1 e r_2 è completo, allora contiene almeno un numero di tuple pari al massimo fra $|r_1|$ e $|r_2|$;

- se $X_1 \cap X_2$ contiene una chiave per r_2 , allora il join di $r_1(X_1)$ e $r_2(X_2)$ contiene al più $|r_1|$ tuple;
- se $X_1 \cap X_2$ coincide con una chiave per r_2 e sussiste il vincolo referenziale fra $X_1 \cap X_2$ in r_1 e la chiave di r_2 , allora il join di $r_1(X_1)$ e $r_2(X_2)$ contiene esattamente $|r_1|$ tuple.

5.3.3 Join esterni

La caratteristica dell'operatore di join di "tralasciare" le tuple di una relazione che non hanno controparte nell'altra è utile in molti casi ma pericolosa in altri in quanto potrebbe omettere informazioni utili. Consideriamo la figura 5.1, supponendo di essere interessati a tutti gli impiegati, con l'indicazione del capo, se noto. Per questo, è stata proposta una variante dell'operatore di join chiamata *join esterno* o *outer join*, che prevede che tutte le tuple diano un contributo al risultato, eventualmente estese con valori nulli ove non vi siano controparti opportune. Esistono tre varianti dell'operatore di join esterno:

1. il join esterno *sinistro*, che estende solo le tuple del primo operando;
2. il join esterno *destra*, che estende solo le tuple del secondo operando;
3. il join esterno *completo*, che le estende tutte.

La seguente figura mostra degli esempi di join esterno.

r_1	<table><tr><th>Impiegato</th><th>Reparto</th></tr><tr><td>Rossi Neri Bianchi</td><td>vendite produzione produzione</td></tr></table>	Impiegato	Reparto	Rossi Neri Bianchi	vendite produzione produzione	r_2	<table><tr><th>Reparto</th><th>Capo</th></tr><tr><td>produzione acquisti</td><td>Mori Bruni</td></tr></table>	Reparto	Capo	produzione acquisti	Mori Bruni
Impiegato	Reparto										
Rossi Neri Bianchi	vendite produzione produzione										
Reparto	Capo										
produzione acquisti	Mori Bruni										
$r_1 \bowtie_{\text{left}} r_2$	<table><tr><th>Impiegato</th><th>Reparto</th><th>Capo</th></tr><tr><td>Rossi Neri Bianchi</td><td>vendite produzione produzione</td><td>null Mori Mori</td></tr></table>			Impiegato	Reparto	Capo	Rossi Neri Bianchi	vendite produzione produzione	null Mori Mori		
Impiegato	Reparto	Capo									
Rossi Neri Bianchi	vendite produzione produzione	null Mori Mori									
$r_1 \bowtie_{\text{right}} r_2$	<table><tr><th>Impiegato</th><th>Reparto</th><th>Capo</th></tr><tr><td>Neri Bianchi null</td><td>produzione produzione acquisti</td><td>Mori Mori Bruni</td></tr></table>			Impiegato	Reparto	Capo	Neri Bianchi null	produzione produzione acquisti	Mori Mori Bruni		
Impiegato	Reparto	Capo									
Neri Bianchi null	produzione produzione acquisti	Mori Mori Bruni									
$r_1 \bowtie_{\text{full}} r_2$	<table><tr><th>Impiegato</th><th>Reparto</th><th>Capo</th></tr><tr><td>Rossi Neri Bianchi null</td><td>vendite produzione produzione acquisti</td><td>null Mori Mori Bruni</td></tr></table>			Impiegato	Reparto	Capo	Rossi Neri Bianchi null	vendite produzione produzione acquisti	null Mori Mori Bruni		
Impiegato	Reparto	Capo									
Rossi Neri Bianchi null	vendite produzione produzione acquisti	null Mori Mori Bruni									

Figura 5.4: Join esterni

5.3.4 Semijoin

Un operatore derivato da quello di join è il *semijoin*, che restituisce le tuple di una relazione che partecipano al join naturale di tale relazione con un'altra. La definizione formale dell'operatore, che viene indicato con il simbolo $\bowtie$, è la seguente:

date due relazioni di $r_1(X_1)$ e $r_2(X_2)$ il semijoin $r_1 \bowtie r_2$ è una relazione definita su X_1 come segue:

$$r_1 \bowtie r_2 = \{t \mid t \in r_1 \text{ ed esiste } t_2 \in r_2 \text{ con } t[X_1 \cap X_2] = t_2[X_1 \cap X_2]\}$$

Sulla base della definizione, è facile osservare che l'operatore può essere espresso per mezzo del join e della proiezione:

$$r_1 \bowtie r_2 = \pi_{X_1}(r_1 \Join r_2)$$

Un esempio di semijoin è mostrato nella seguente figura:

r_1	<table><tr><th>Impiegato</th><th>Reparto</th></tr><tr><td>Rossi</td><td>vendite</td></tr><tr><td>Neri</td><td>produzione</td></tr><tr><td>Bianchi</td><td>produzione</td></tr></table>	Impiegato	Reparto	Rossi	vendite	Neri	produzione	Bianchi	produzione	r_2	<table><tr><th>Reparto</th><th>Capo</th></tr><tr><td>produzione</td><td>Mori</td></tr><tr><td>acquisti</td><td>Bruni</td></tr></table>	Reparto	Capo	produzione	Mori	acquisti	Bruni
Impiegato	Reparto																
Rossi	vendite																
Neri	produzione																
Bianchi	produzione																
Reparto	Capo																
produzione	Mori																
acquisti	Bruni																
$r_1 \bowtie r_2$	<table><tr><th>Impiegato</th><th>Reparto</th></tr><tr><td>Neri</td><td>produzione</td></tr><tr><td>Bianchi</td><td>produzione</td></tr></table>	Impiegato	Reparto	Neri	produzione	Bianchi	produzione										
Impiegato	Reparto																
Neri	produzione																
Bianchi	produzione																

Figura 5.5: Semijoin

5.3.5 Join n-ario, intersezione e prodotto cartesiano

Vediamo alcune proprietà dell'operatore di join naturale. In primo luogo, osserviamo che l'operatore di join è commutativo e associativo. Pertanto, possiamo scrivere, ove necessario, sequenza di join senza parentesi:

$$r_1 \Join r_2 \Join \dots \Join r_n \text{ oppure } \Join_{i=1}^n r_i$$

Se gli insiemi degli attributi, X_1 e X_2 , sono uguali o disgiunti si applica la definizione generale vista sopra, ma in particolare, se $X_1 = X_2$ allora il join coincide con l'intersezione:

$$r_1(X_2) \Join r_2(X_2) = r_1(X_1) \cap r_2(X_2) \iff X_1 = X_2$$

Il caso in cui gli insiemi degli attributi sono disgiunti merita ancora maggiore attenzione. Il risultato è sempre definito sull'unione X_1X_2 , e ciascuna tupla deriva sempre da due tuple, una per ciascuno degli operandi, ma dato che le tuple non hanno attributi comuni, non viene richiesta a esse nessuna condizione per partecipare insieme al join. Quindi, il risultato del join contiene le tuple ottenute combinando, in tutti i modi possibili, le tuple degli operandi. In questo caso allora il join diventa un *prodotto cartesiano*.

Impiegati		Progetti	
Impiegato	Progetto	Codice	Nome
Rossi	A	A	Venere
Neri	A	B	Marte
Neri	B		

Impiegati $\bowtie$ Progetti			
Impiegato	Progetto	Codice	Nome
Rossi	A	A	Venere
Neri	A	A	Venere
Neri	B	A	Venere
Rossi	A	B	Marte
Neri	A	B	Marte
Neri	B	B	Marte

Figura 5.6: Un prodotto cartesiano

5.3.6 Theta-join ed equi-join

Osservando la figura 5.6 possiamo notare che un prodotto cartesiano di solito ha poca utilità, in quanto concatena tuple non correlate dal punto di vista semantico. Spesso, il prodotto cartesiano viene seguito da una selezione, che c'entra l'attenzione su tuple correlate secondo le esigenze. Per esempio, sulle relazioni *Impiegati* e *Progetti* ha senso definire un prodotto cartesiano seguito dalla selezione che mantiene solo le tuple con valori uguali sull'attributo **Progetto** di *Impiegati* e su **Codice** di *Progetti*, come mostrato nella seguente figura:

Impiegati		Progetti	
Impiegato	Progetto	Codice	Nome
Rossi	A	A	Venere
Neri	A	A	Venere
Neri	B	B	Marte

$\sigma_{\text{Progetto}=\text{Codice}}(\text{Impiegati} \bowtie \text{Progetti})$			
Impiegato	Progetto	Codice	Nome
Rossi	A	A	Venere
Neri	A	A	Venere
Neri	B	B	Marte

Figura 5.7: Un prodotto cartesiano seguito da una selezione

Per questo viene definito un operatore derivato, il *theta-join*, come prodotto cartesiano seguito da una selezione, nel modo seguente:

$$r_1 \bowtie_F r_2 = \sigma_F(r_1 \bowtie r_2)$$

ove F è una formula proposizionale. La relazione nella figura 5.7 può essere ottenuta per mezzo del theta-join:

$$\text{Impiegati} \bowtie_{\text{Progetto}=\text{Codice}} \text{Progetti}$$

Un theta-join in cui la condizione è una congiunzione di atomi di uguaglianza viene chiamato *equi-join*.

Il theta-join, in particolare l'equi-join, hanno un importante risvolto pratico, in quanto la maggior parte dei sistemi di basi di dati non utilizzano i nomi di attributi per correlare relazioni e quindi non utilizzano il join naturale ma il theta-join e l'equi-join. Il join naturale può essere simulato per mezzo della ridenominazione, dell'equi-join e della proiezione.

Esempio : Simulazione join naturale

Date due relazioni $r_1(ABC)$ e $r_2(BCD)$, il join naturale di r_1 e r_2 può essere espresso per mezzo degli altri operatori, in tre passi:

- ridenominando gli attributi in modo di ottenere relazioni su schemi disgiunti:

$$\rho_{B'C' \leftarrow BC}(r_2)$$

- effettuando l'equi-join, con condizioni di uguaglianza sugli attributi corrispondenti:

$$r_1 \bowtie_{B=B' \wedge C=C'} \rho_{B'C' \leftarrow BC}(r_2)$$

- concludendo con una proiezione che elimina gli attributi “doppioni”, che presentano valori identici a quelli di altri attributi:

$$\pi_{ABCD}(r_1 \bowtie_{B=B' \wedge C=C'} \rho_{B'C' \leftarrow BC}(r_2))$$